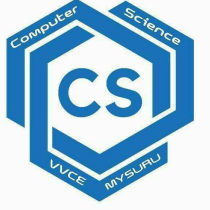


Vidyavardhaka College of Engineering, Mysuru

Autonomous Institute, Affiliated to VTU

Accredited by NBA | NAAC with 'A' Grade



Module 3 : Syntax Analysis II

Introduction to LR parsing

- LR(k) parsing is another technique to perform bottom- up parsing.
- LR(k) will parse the input string by applying rightmost derivation in the reverse.
- LR(k), where
 - L stands for Left to right scanning of the input.
 - R stands for rightmost derivation in the reverse.
 - K stands for no of input symbols of lookahead that are used in making parsing decisions.

Why LR parsers?

- LR parsers are table driven, much like non recursive LL parsers.
- A grammar for which we can construct a parsing table using one of the LR parsing Method
 - ❖ SLR (simple LR)
 - ❖ CLR (Canonical LR)
 - ❖ LALR (Lookahead LR)
- For a grammar to be LR it is sufficient that a left-to-right shift reduce parsers be able to recognize handles of right sentential forms when they appears on the top of the stack.

LR parsing Advantages

- LR parsers can be constructed to recognize virtually all programming language constructs for which context free grammars can be written.
- LR parsing method is the most generally non- backtracking shift reduce parsing method.
- LR parsing method can detect a syntactic error as soon as it starts left to right scan of input.
- LR parsers can describe more languages than LL grammars.

LR parsing Disadvantages

- The main drawback is they consumes too much time to construct by hand for a typical programming language grammars.
- This disadvantages can be eliminated using LR parser generators such as YACC.

Types of LR Parsers

1. Simple LR Parsers(SLR):

- Easy to implement
- But , sometimes it may fail to produce the parsing table for certain grammars.

2. Canonical LR parsers: (CLR)

- More powerful and works on very large class of grammars.
- Very expensive to implement.

3. Look-Ahead LR parsers: (LALR)

- Intermediate between SLR and CLR.
- Implemented for most of the grammars.

Items and the LR(0) Automaton

□ An LR(0) item or item of a grammar G is a production of G with a dot(.) on the right hand side of production.

□ example:

Consider the production $A \rightarrow BCD$

The various items that can be obtained using this production are:

$A \rightarrow .BCD$

$A \rightarrow B.CD$

$A \rightarrow BC.D$

$A \rightarrow BCD.$

Note: $A \rightarrow \epsilon$ (epsilon) then item is $A \rightarrow .$

The all above items are called Canonical LR(0) collections , we can construct a DFA.

Canonical LR(0)

- The construction of a Canonical LR(0) collection of items for a grammar can be obtained using:

1. An augmented grammar
2. Closure function
3. GOTO function
4. Items Function.

Augmented grammar

- Let $G = (V, T, S, P)$ be a grammar an augmented G' for the grammar G is defined as $G' = (V', T, S', P')$.
- We are adding a new production for the start symbol i.e., $S' \rightarrow S$
- The main purpose of the new starting production is to indicate to the parser when it should stop parsing and announce acceptance of the input.

• Example :

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

Augmented grammar

$E' \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

Closure of Item sets:

□ If I is a set of items for a grammar G then $CLOSURE(I)$ is constructed using the following two rules.

Rule 1: Initially, add every item in I to $CLOSURE(I)$.

Rule 2: If $A \rightarrow \alpha . B \beta$ is in $CLOSURE(I)$ and $B \rightarrow \gamma$ is a production and $B \rightarrow . \Gamma$ is not there in $CLOSURE(I)$, then add $B \rightarrow . \Gamma$ to $CLOSURE(I)$.

apply rule 2 repeatedly until no more new items are added to $CLOSURE(I)$.

Consider an augmented grammar then the closure items set is written as

$E' \rightarrow . E$

$E \rightarrow . E + T$

$E \rightarrow . T$

$T \rightarrow . T * F$

$T \rightarrow . F$

$F \rightarrow . (E)$

$F \rightarrow . id$

```

SetOfItems CLOSURE( $I$ ) {
     $J = I$ ;
    repeat
        for ( each item  $A \rightarrow \alpha \cdot B \beta$  in  $J$  )
            for ( each production  $B \rightarrow \gamma$  of  $G$  )
                if (  $B \rightarrow \cdot \gamma$  is not in  $J$  )
                    add  $B \rightarrow \cdot \gamma$  to  $J$ ;
    until no more items are added to  $J$  on one round;
    return  $J$ ;
}

```

Figure 4.32: Computation of CLOSURE

GOTO Function

□ The $\text{GOTO}(I, X)$ is a function which accepts set of items I as the first arguments and X which is a grammar symbol which can be either terminal or a non-terminal as the second arguments.

□ It is defined as ,

if $A \rightarrow \alpha . X \beta$ in I

$\text{GOTO}(I, X) = \text{CLOSURE}(A \rightarrow \alpha X . \beta)$

Computation of the canonical collection of sets of LR(0) items

```
void items( $G'$ ) {  
     $C = \text{CLOSURE}(\{[S' \rightarrow \cdot S]\})$ ;  
    repeat  
        for ( each set of items  $I$  in  $C$  )  
            for ( each grammar symbol  $X$  )  
                if (  $\text{GOTO}(I, X)$  is not empty and not in  $C$  )  
                    add  $\text{GOTO}(I, X)$  to  $C$ ;  
    until no new sets of items are added to  $C$  on a round;  
}
```

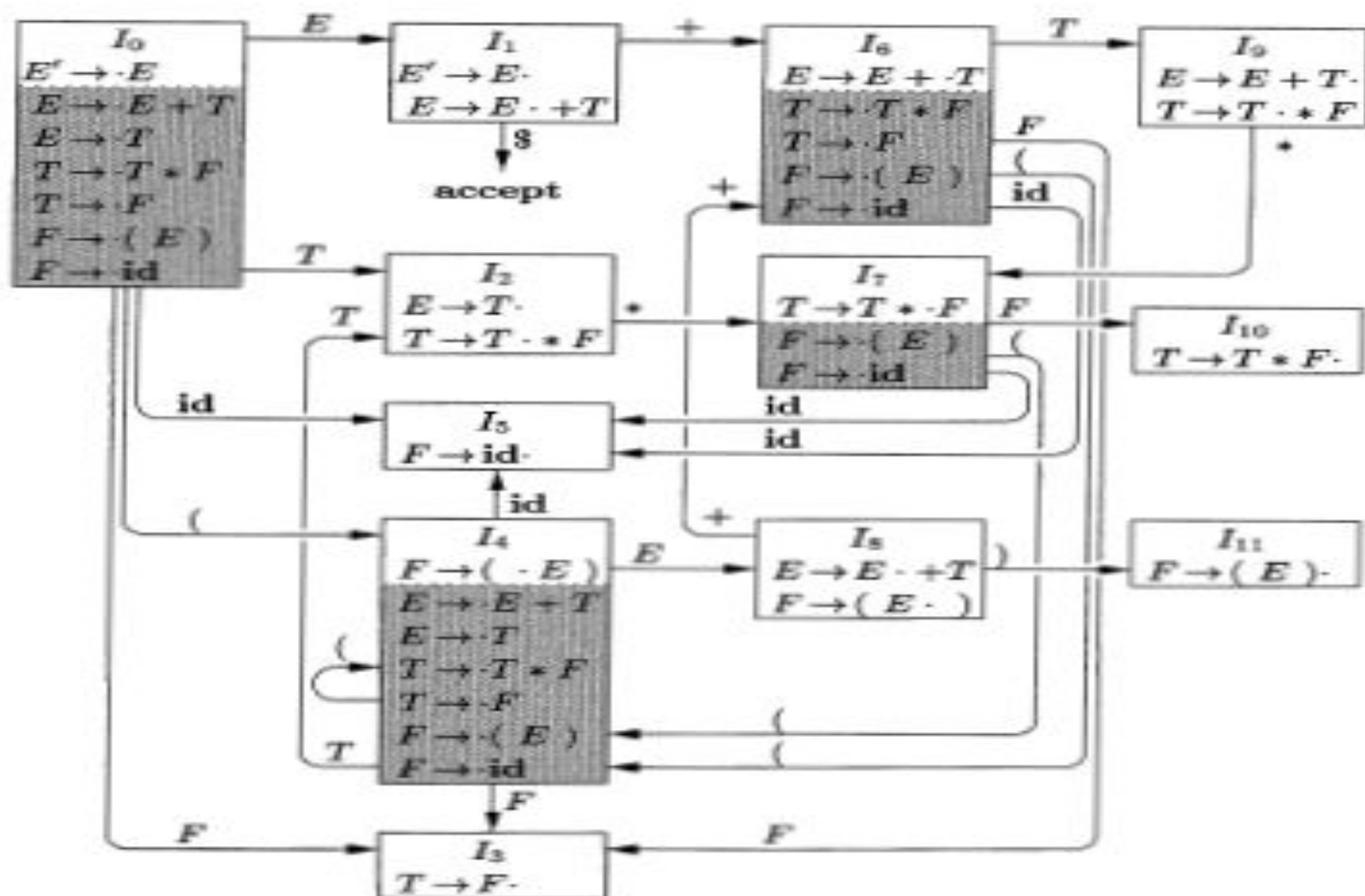


Figure 4.31: LR(0) automaton for the expression grammar (4.1)

STATE	ACTION						GOTO		
	id	+	*	(	)	\$	<i>E</i>	<i>T</i>	<i>F</i>
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Figure 4.37: Parsing table for expression grammar

LR Parsing Algorithm

- It consists of an input , output , a stack , a driver program and a parsing table has 2 parts (ACTION and GOTO).
- The parsing program reads a characters from an input buffer one at a time, where a shift – reduce parsers would shift a symbol , an LR parsers shift a state.

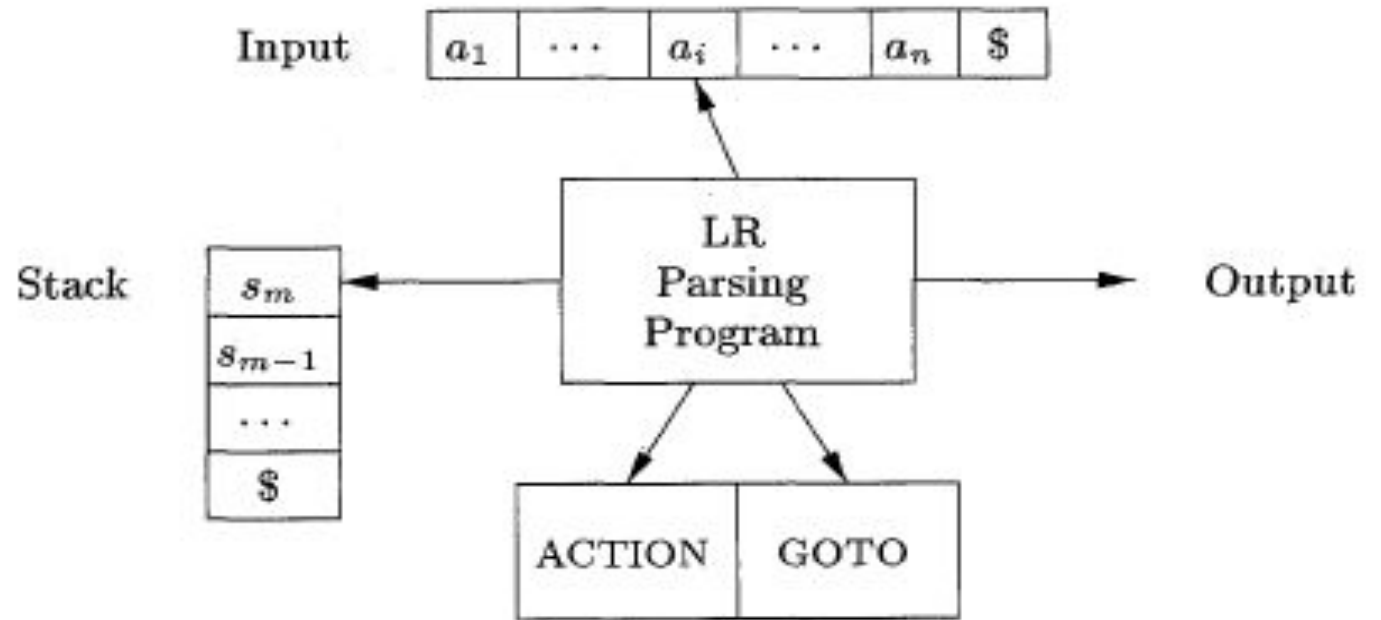


Figure 4.35: Model of an LR parser

Structure of the LR Parsing Table

The parsing table consists of two parts: a parsing-action function ACTION and a goto function GOTO.

1. The ACTION function takes as arguments a state i and a terminal a (or $\$,$ the input endmarker). The value of ACTION[i, a] can have one of four forms:
 - (a) Shift j , where j is a state. The action taken by the parser effectively shifts input a to the stack, but uses state j to represent a .
 - (b) Reduce $A \rightarrow \beta$. The action of the parser effectively reduces β on the top of the stack to head A .
 - (c) Accept. The parser accepts the input and finishes parsing.
 - (d) Error. The parser discovers an error in its input and takes some corrective action. We shall have more to say about how such error-recovery routines work in Sections 4.8.3 and 4.9.4.
2. We extend the GOTO function, defined on sets of items, to states: if GOTO[I_i, A] = I_j , then GOTO also maps a state i and a nonterminal A to state j .

Chapter 5

Syntax-Directed Translation

Introduction

- The main goal of the semantic analysis is to check the correctness of program and enable proper execution.
- Semantic analysis is the third phase of the compiler which acts as an interface between syntax analysis phase and Intermediate code generation phase.
- It accepts the parse tree from the syntax analysis phase and adds the semantic information to the parse tree and performs certain checks based on this information.

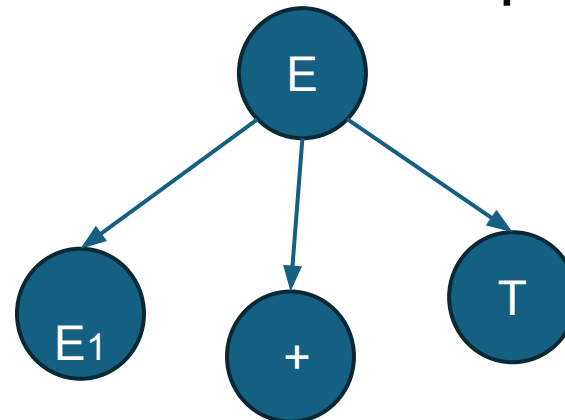
□ The semantics of a languages can be described using two notations namely

1. Syntax Directed Definition (SDD)
2. Syntax Directed Translations (SDT)

Note: Consider the production $E \rightarrow E + T$, to distinguish E on LHS and E on RHS we will use E_1 on RHS of production then ,

$E \rightarrow E_1 + T$

parse Tree



Syntax Directed Translation

- The most general approach to syntax directed translation is to construct a parse tree or a syntax tree and then to compute the values of a attributes at the node of the tree by visiting the nodes of the tree.
- Two types of SDT
 1. L – attributed translations-- (L stands for Left to right)
 - It consists of both synthesized and inherited attributes
 2. S – attributed translations --- (S for synthesized)
 - It consists of synthesized attributes
 - It can be performed easily with bottom-up parse.

Semantic Rules

- The Rule that describes how to compute the attribute values of the attributes associated with a grammar symbols using attribute values of other symbols is called semantic rule.
- Example :

$$\mathbf{E.val = E1.val + T.val}$$

Syntax Directed Definition (SDD)

- A syntax Directed definition (SDD) is a context free grammar with attributes and semantic rules.
- The attributes are associated with grammar symbols and semantic rules are associated with Productions.
- Consider the SDD for the production

$$E \rightarrow E_1 + T$$

Production	Semantic Rules
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$

SDD

- An attribute is a property of a programming language construct.
- Attributes are always associated with a grammar symbols.
- If X is a grammar symbol and ' α ' is a attribute , then $X.\alpha$ is a value of attribute ' α ' at a particular node X in the parse tree.

□ Examples:

1. val = attribute

E = Non-terminal, then node E is represented as E.val

2. lexval = attribute

digit = Terminal, the node digit represented as digit.lexval

Attributes

- Typical Examples of an attributes are

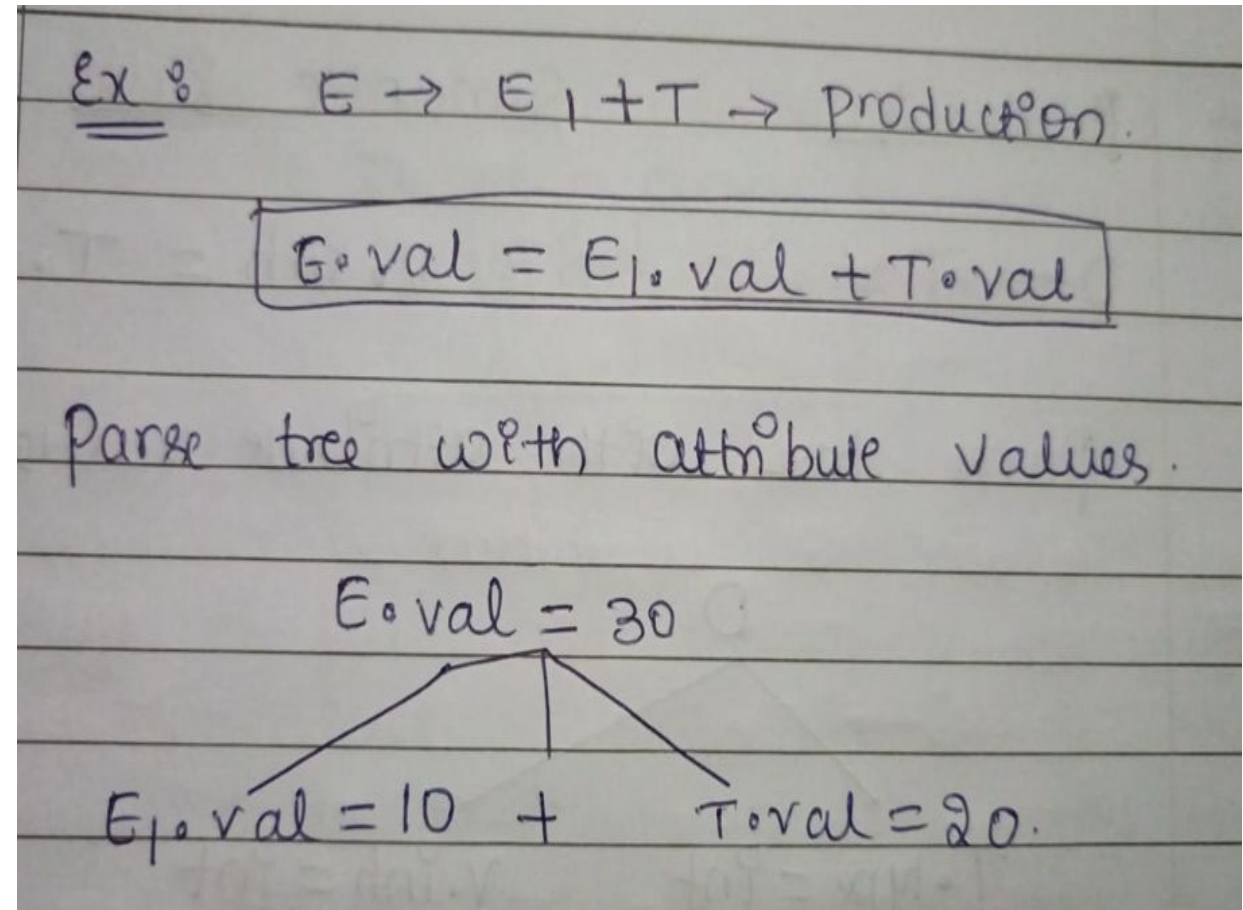
1. Data types associated with variables such as int, float, char.
2. Value of an expression e.g. E.val
3. The location of a variable in memory
4. The object code of a function or procedure.
5. The number of significant digits in a number and so on.

Types

- The attribute value for a node in the parse tree may depend on information from its children nodes or its sibling nodes or parent nodes.
- There are 2 types of attributes for the non terminals are
 1. Synthesized attribute (S-attributes)
 2. Inherited attribute (I – attribute)

Synthesized attributes

- A *synthesized attribute*: the attribute value of a non-terminal A derived from the attribute values of its children or itself.
- The attribute values of synthesized attributes are passed up from children to the parent node in bottom-up manner.

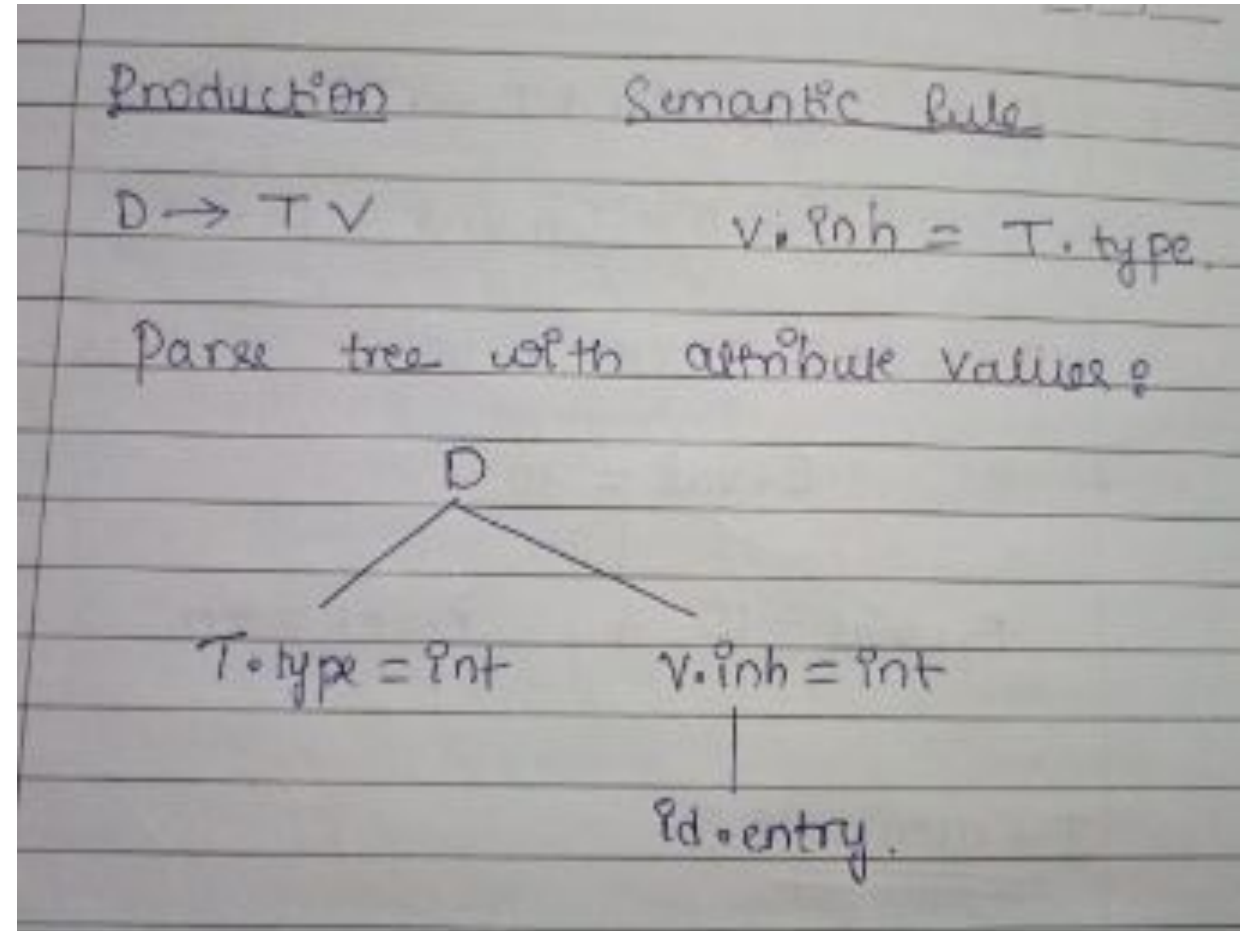


Inherited- attribute

- An *inherited attribute*: the attribute value of a non-terminal A derived from the attribute values of its siblings or from its parents or itself.
- The attribute values of inherited attributes are passed from siblings or from parent to children in top-down manner.
- Example : $D \rightarrow TV$

Single declaration :- $D = \text{int Sum}$

- Terminals can have synthesized attributes but not inherited attributes.
- Attributes for a terminal have lexical values that are supplied by the lexical analyzers.



Annotated Parse tree

- A parse tree showing the attribute values of each node is called annotated parse tree.
- The terminals in annotated parse tree can have only synthesized attribute values and are obtained directly from the lexical analyzer.

1. Write the SDD for a simple Desk calculator and show annotated parse tree for expression $3 * 5 + 4 n$

PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F $	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

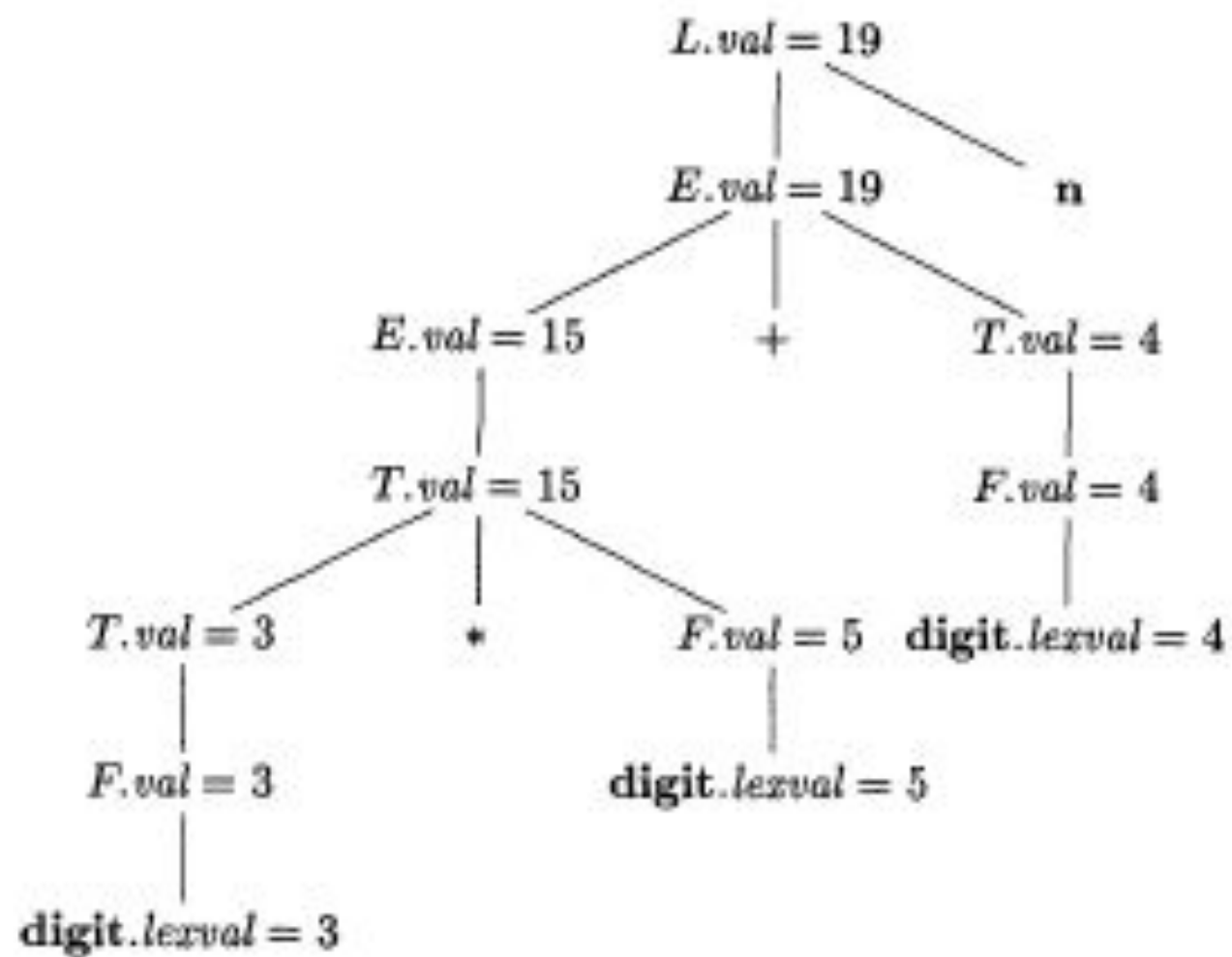


Figure 5.3: Annotated parse tree for $3 * 5 + 4n$

2. Write the SDD for a simple Desk calculator and show annotated parse tree for expression(3 + 4) * (5 + 6) n

Given grammar is

$L \rightarrow En$

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow (E) \mid \text{digit}$

Productions

$$S \rightarrow E \cap$$

$$E \rightarrow E + T$$

$$E \rightarrow E - T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow T / F$$

$$T \rightarrow F$$

$$F \rightarrow (E)$$

$$F \rightarrow \text{digit}$$

Semantic Rules

$$S \cdot \text{val} = E \cdot \text{val}$$

$$E \cdot \text{val} = E_1 \cdot \text{val} + T \cdot \text{val}$$

$$E \cdot \text{val} = E_1 \cdot \text{val} - T \cdot \text{val}$$

$$E \cdot \text{val} = T \cdot \text{val}$$

$$T \cdot \text{val} = T_1 \cdot \text{val} * F \cdot \text{val}$$

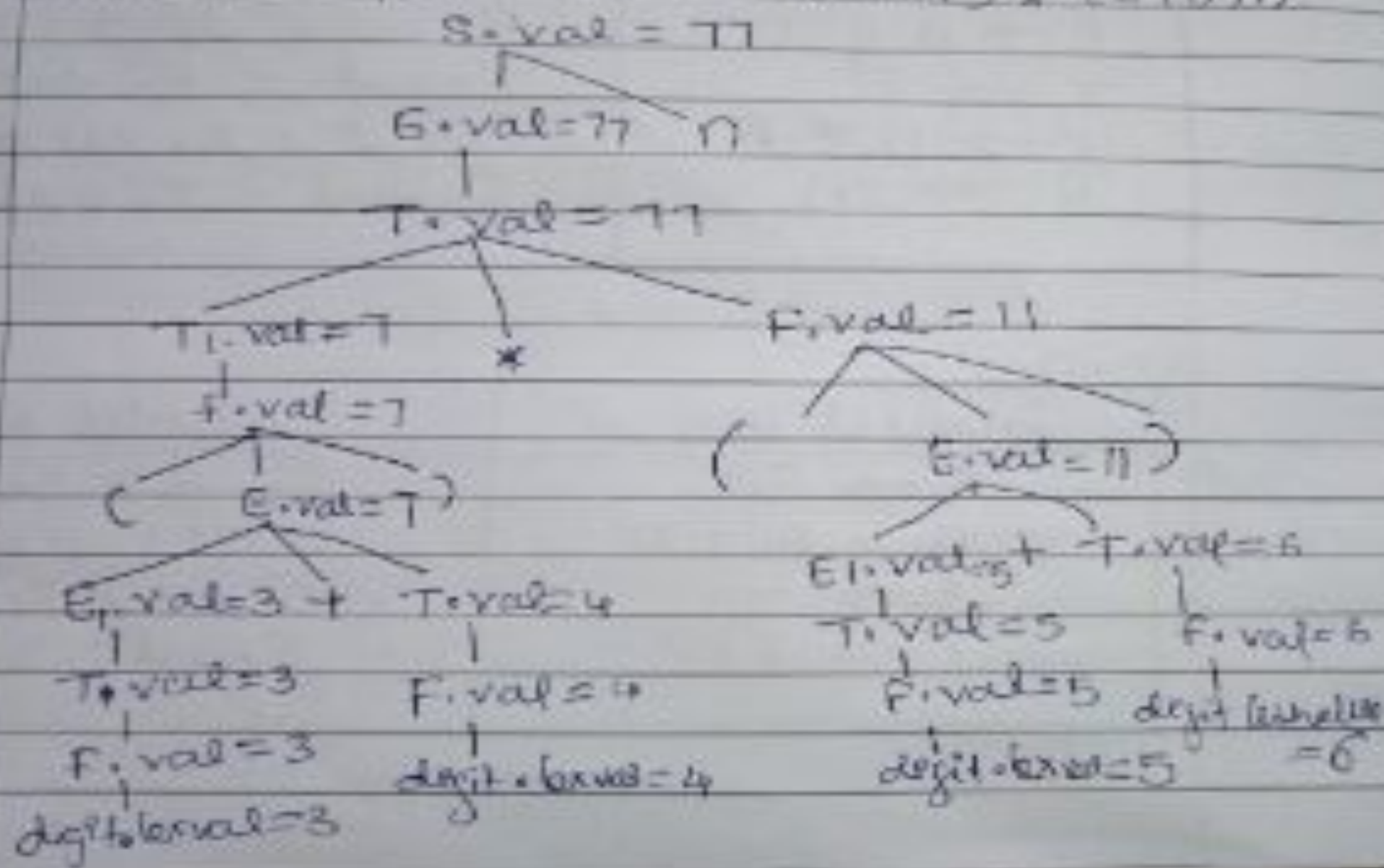
$$T \cdot \text{val} = T_1 \cdot \text{val} / F \cdot \text{val}$$

$$T \cdot \text{val} = F \cdot \text{val}$$

$$F \cdot \text{val} = E \cdot \text{val}$$

$$F \cdot \text{val} = \text{digit} \cdot \text{lexval}$$

Annotated parse tree $\Rightarrow (3+4) * (5+6) 0$



Evaluating an SDD at the nodes of a Parse Tree

- We can easily obtain an SDD using the following steps.

Step 1 : Construct the parse tree

Step 2 : Use the rules to evaluate attributes of all the nodes of the parse tree.

Step 3: Obtain the attribute values for each non-terminals and write the semantic rules for each production. When complete annotated parse tree is ready, we will have the complete SDD.

Circular dependency

- If the attribute value of a parent node depends on the attribute value of a child node and vice-versa, then we say there exists a circular dependency between the child node and parent node.
- In this situation, it is not possible to evaluate the attribute of either parent node or the child node since one value depends on another value.

PRODUCTION

$$A \rightarrow B$$

SEMANTIC RULES

$$A.s = B.i;$$

$$B.i = A.s + 1$$

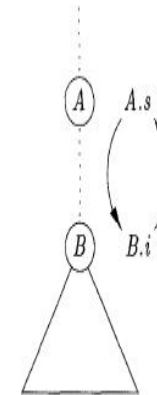


Figure 5.2: The circular dependency of $A.s$ and $B.i$ on one another

Why we have to evaluate inherited attributes

- Consider the following grammar

$T \rightarrow T * F \mid F$

$F \rightarrow \text{digit}$

- ❖ Using the above productions, we can generate an un-parenthesized expression consisting of only $*$ operator.
- ❖ The above grammar has left -recursion and it is suitable for bottom-up parser such as LR parsers.
- Consider annotated parse tree for evaluating the expression $3 * 5$ is shown below.

Example : Obtain SDD for the following grammar using Top-down approach for the expression $3 * 3$

Grammar : $T \rightarrow T * F \mid F$
 $F \rightarrow \text{digit}$

Solution:

- ❖ The given grammar has left recursion and hence it is not suitable for top-down parsers.
- ❖ After removing left recursion , the grammar is written as

$$T \rightarrow F T'$$
$$T' \rightarrow * F T' \mid \epsilon$$

PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

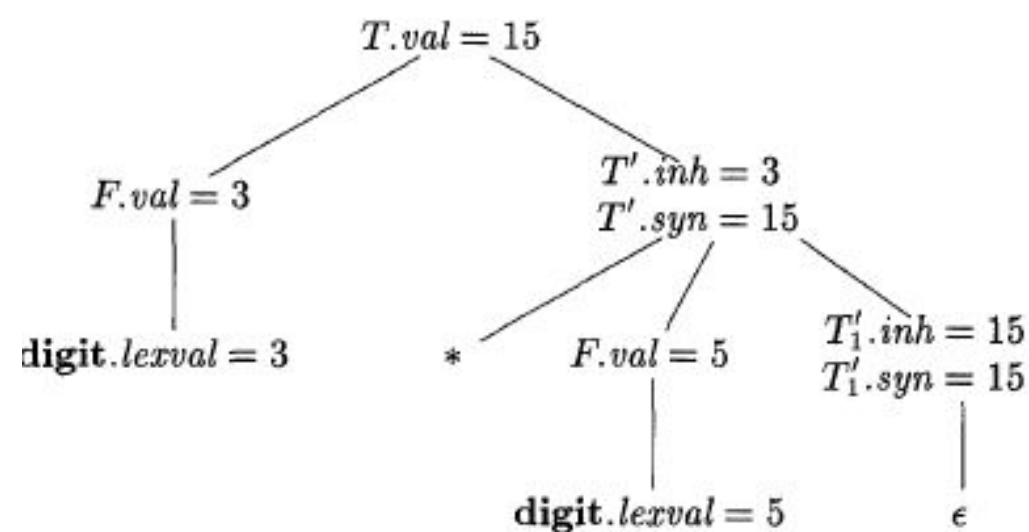


Figure 5.5: Annotated parse tree for $3 * 5$

Evaluation Orders for SDD

- ❖ Dependency Graphs
- ❖ Ordering the evaluation of attributes
- ❖ S-attributed definition
- ❖ L-attributed definition
- ❖ Semantic rules with controlled side effects.

Evaluation orders for SDD's

- The evaluation order to find attribute values in a parse tree using semantic rules can be easily obtained with the help of dependency graph.
- While annotated parse tree shows the values of attributes, a dependency graph helps us to determine how those values can be computed.

Dependency Graphs

- *A dependency graph* depicts the flow of information among the attribute instances in a particular parse tree;
- An edge from one attribute instance to another means that the value of the first is needed to compute the second.

• Example:

PRODUCTION

$E \rightarrow E_1 + T$

SEMANTIC RULE

$E.val = E_1.val + T.val$

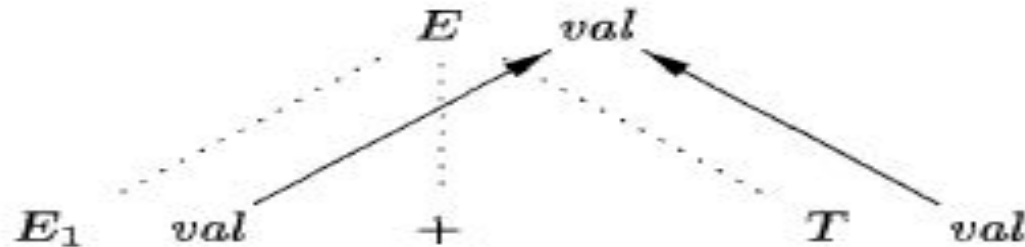


Figure 5.6: $E.val$ is synthesized from $E_1.val$ and $E_2.val$

Example

PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

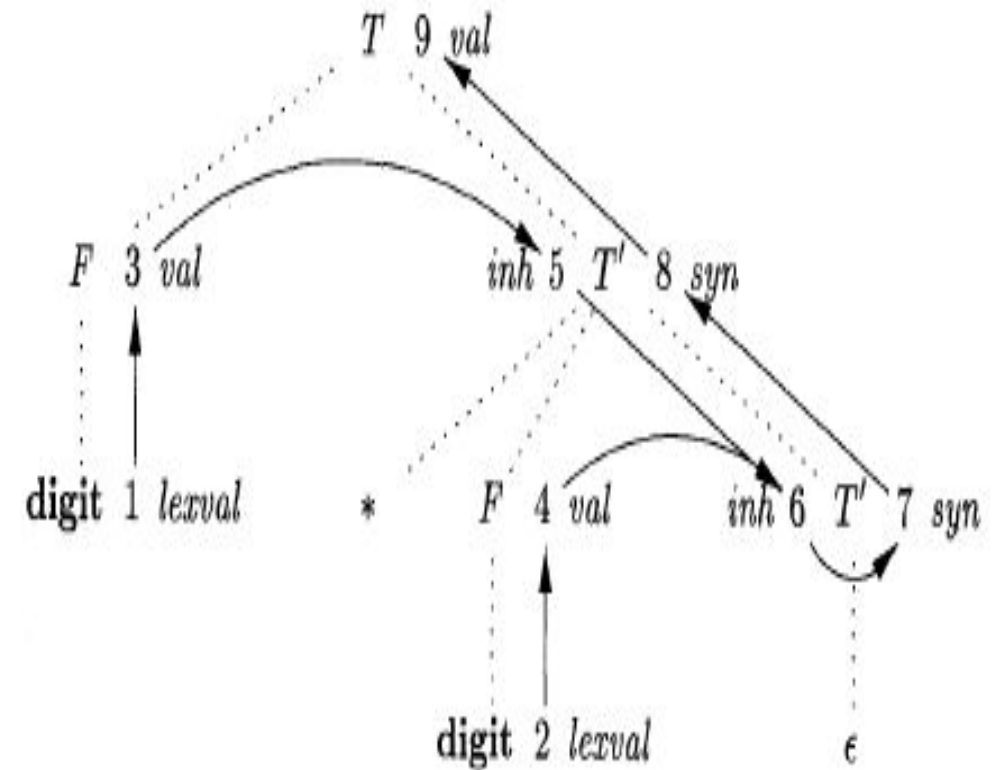


Figure 5.7: Dependency graph for the annotated parse tree of Fig. 5.5

Ordering the evaluation of attributes

- The dependency graph characterizes the possible orders in which we can evaluate the attributes at the various nodes of a parse tree.
- Ordering embeds a directed graph into a linear order and is called a topological sort of the graph.

S-Attributed Definition:

- Given an SDD, it is very difficult to tell whether there exists a cycle in the dependency graph corresponding a parse tree.
- But, in practice, translations can be implemented using classes of SDD's that will guarantee an evaluation order, since they do not permit dependency graphs with cycles.
- The 2 classes of SDD's that guarantee an evaluation order are:
 - a. S-attributed definition
 - b. L-attributed definition

- **S-Attributed definition:** An SDD is S-attributed if every attribute is synthesized.
- In an S-attributed SDD, each semantic rule computes the attribute value of a non terminal that occurs on LHS of the production that represents head of the production with the help of the attribute values of non terminals on the right hand side of the production that represent body of the production

PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

- **L-attributed definition:**

- the idea behind this class is that, between the attributes associated with the production body, dependency graph it just can go from left to right but not from right to left (hence L-attributed).
- More-precisely, each attribute must be either:
 - a. Synthesized, or
 - b. Inherited.

Chapter-6

Intermediate-Code Generator

Variants of Syntax Trees:

- Nodes in a syntax tree represent constructs in the source program; the children of a node represent the meaningful components of a construct.
- A **directed acyclic graph**(called a **DAG**) for an expression identifies the common 'subexpressions' of the expression.
- DAG's can be constructed by using the same techniques that construct syntax trees.

Directed Acyclic Graphs for Expressions

- Like the syntax tree for an expression, an DAG has leaves corresponding to atomic of parents and interior nodes corresponding to operators.
- The difference is that a node N in a DAG has more than one parent if N represents a common subexpression; in a syntax tree, the tree for a common sub expression would be replicated as many times as the expression appears in the original expression.
- Thus, a DAG not only represents expressions more succinctly, it gives the compiler important clues regarding the generation of efficient code to evaluate the expressions.

- The SDD Can construct either syntax trees or DAG's.
- it was used to construct syntax trees, where functions 'Leaf' and 'Node' created a fresh node each time they were called.
- It will construct a DAG if, before creating a new node, these functions first check whether an identical node already exists.
- If a previously created identical node exists, the existing node is returned.
- For instance, before constructing a new node, **Node(op, left, right)** we check whether there is already a node with label op, and children 'left' and 'right', in that order.
- If so, 'Node' returns the existing node; otherwise, it creates a new node.

Three Address Code:

- It is an intermediate code used by the optimising compilers.
- In three address code, the given expression is broken down into several separate instructions.
- These instructions are easily translated into assembly language.
- Each three address code has atmost 3 operands.
- It is a combination of assignment and a binary operator.
- In three address code, there is atmost one operator on the right hand side of the instruction.
- Example: $x+y*z$

Different Representations of Three Address Code:

- Three address code can be represented in various forms viz. **quadruples, triples and indirect triples.**
1. **Quadruples:** It will be constructed by consulting every production during the construction of three address code. It consists of 4 fields namely, **op, arg1, arg2, and result.**
 2. **Triples:** It will be constructed by consulting quadruple representation and we have to write references. It consists of 3 fields namely, **op, arg1, and arg2.**

3. Indirect Triples: It makes use of pointers to the listing of all references to the competitions which is made separately and stored. it requires less space than triplet representation.